

Test session (A)

A.1 To test during test session

- Keyboard layouts
- bashrc
- vimrc
- Makefile
- template.cpp
- Python
- Printing
- Sending clarifications
- Documentation
- Whitespace/case insensitivity in output
- Source code limit
- How long do array accesses take on machine resp. judge?
- How long do small operations take on machine resp. judge?
- Pragmas
- Verdicts:
 - Accepted
 - WA
 - Presentation Error
 - RTE (what results in RTE?)
 - TLE
 - Memory limit, both stack and heap
 - Output limit
 - Compile error

A.2 Warm-up diagnostics

A.2.1 Performance

warmup.cpp

Description: Benchmarks common C++ operations during the warm-up session.

Usage: Change TEST and the parameters below. TEST: 0=integer, 1=double, 2=sequential array, 3=dependent array, 4=recursion.

3ada71, 79L, 6m50s

```
constexpr int TEST = 0;

constexpr long long N = 100000000;
constexpr int ARRAY_LOG = 20;
constexpr int REC_DEPTH = 100;
constexpr int REC_RUNS = 100000;

using ull = unsigned long long;

__attribute__((noinline))
ull recurse(int depth, ull x) {
    volatile ull y = x + depth;
    if (!depth) return y;
    ull ans = recurse(depth - 1, y);
    return ans + (y & 1);
} // a1934b

volatile ull sink_int;
volatile double sink_double;

void test() {
    const size_t A = 1ULL << ARRAY_LOG;
    const size_t MASK = A - 1;

    vector<ull> a;

    if (TEST == 2) {
        a.resize(A);
        iota(a.begin(), a.end(), 1);
    } // d4f8f4
    if (TEST == 3) {
        a.resize(A);
        for (size_t i = 0; i < A; i++) a[i] = (5 * i + 1) &
            MASK;
    } // bafa1d

    auto start = chrono::steady_clock::now();

    if (TEST == 0) {
        ull x = 123456789;
        for (long long i = 0; i < N; i++) {
            x ^= x << 7;
            x ^= x >> 9;
            x += i;
        } // bee91b
        sink_int = x;
    } // 2ee310
```

```
else if (TEST == 1) {
    double x = 1.0000001;
    for (long long i = 0; i < N; i++) x = x * 1.0000001
        + 0.0000001;
    sink_double = x;
} // 55c070
else if (TEST == 2) {
    ull x = 0;
    for (long long i = 0; i < N; i++)x += a[i & MASK];
    sink_int = x;
} // 47ae6d
else if (TEST == 3) {
    ull x = 0;
    size_t pos = 0;
    for (long long i = 0; i < N; i++) {
        pos = a[pos];
        x += pos;
    } // 002e72
    sink_int = x;
} // cbfa0c
else if (TEST == 4) {
    ull x = 0;
    for (int i = 0; i < REC_RUNS; i++)x += recurse(
        REC_DEPTH, i);
    sink_int = x;
} // 147e80

auto end = chrono::steady_clock::now();
double sec = chrono::duration<double>(end - start).
    count();
long long ops = TEST == 4 ? 1LL * REC_RUNS * (
    REC_DEPTH + 1) : N;
cerr << fixed << setprecision(3);
cerr << sec << " s, " << sec * 1e9 / ops << " ns/op\n
";
if (TEST == 1) cout << sink_double << '\n';
else cout << sink_int << '\n';
} // 0547f9
```

pywarmup.py

Description: Benchmarks common Python operations during the warm-up session.

Usage: Change TEST and the parameters below. TEST: 0=integer, 1=float, 2=sequential list, 3=dependent list, 4=recursion.

56L, 4m30s

```
import sys, time

TEST = 0
N = 10_000_000
ARRAY_LOG = 18
REC_DEPTH = 100
REC_RUNS = 10_000

def recurse(depth, x):
    y = x + depth
    if depth == 0: return y
```

```
ans = recurse(depth - 1, y)
return ans + (y & 1)

A = 1 << ARRAY_LOG
MASK = A - 1
a = None
if TEST == 2:
    a = list(range(A))
elif TEST == 3:
    a = [(5 * i + 1) & MASK for i in range(A)]
elif TEST == 4:
    sys.setrecursionlimit(max(1000, REC_DEPTH + 100))

start = time.perf_counter()

x = 0
if TEST == 0:
    x = 123456789
    for i in range(N):
        x ^= x << 7
        x ^= x >> 9
        x += i
        x &= 0xffffffffffffffff
elif TEST == 1:
    x = 1.000001
    for _ in range(N):
        x = x * 1.0000001 + 0.0000001
elif TEST == 2:
    for i in range(N): x += a[i & MASK]
elif TEST == 3:
    pos = 0
    for _ in range(N):
        pos = a[pos]
        x += pos
elif TEST == 4:
    for i in range(REC_RUNS): x += recurse(REC_DEPTH, i
)

elapsed = time.perf_counter() - start
ops = REC_RUNS * (REC_DEPTH + 1) if TEST == 4 else N
print(x)
print(
    f"{elapsed:.3f} s, "
    f"{elapsed * 1e9 / ops:.1f} ns/op",
    file=sys.stderr,
```

A.2.2 Limits and Runtime Errors

```
errors.cpp
Description: Tests memory/stack limits and common runtime failures.
Usage:      Change TEST and parameters.  TEST: 0=vector
allocation, 1=local array, 2=finite recursion, 3=vector push.back,
4=queue push, 5=out of bounds, 6=infinite recursion.
c8c1d8, 68L, 6m50s

constexpr int TEST = 0;
constexpr size_t HEAP_MB = 1024;
constexpr size_t STACK_MB = 128;
constexpr int REC_DEPTH = 10000;
constexpr size_t FRAME_KB = 1;
constexpr size_t BAD_INDEX = 1000000000ULL;
volatile long long sink;
volatile bool stop_recursion = false;

void touch(volatile unsigned char* a, size_t n) {
    for (size_t i = 0; i < n; i += 4096) a[i] = 1;
    if (n) a[n - 1] = 1;
} // 726abf

void local_array_test() {
    volatile unsigned char a[STACK_MB * 1024 * 1024];
    touch(a, sizeof(a));
    sink = a[sizeof(a) - 1];
} // 57c009

__attribute__((noinline))
long long finite_recursion(int depth) {
    volatile unsigned char frame[FRAME_KB * 1024];
    frame[0] = depth;
    frame[sizeof(frame) - 1] = depth;
    if (!depth) return frame[0];
    long long ans = finite_recursion(depth - 1);
    return ans + frame[depth % sizeof(frame)];
} // 2f56f4

__attribute__((noinline))
void infinite_recursion(int depth) {
    volatile unsigned char frame[FRAME_KB * 1024];
    frame[0] = depth;
    frame[sizeof(frame) - 1] = depth;
    if (stop_recursion) return;
    infinite_recursion(depth + 1);
    sink += frame[depth % sizeof(frame)];
} // 7c7c53

void test() {
    const size_t bytes = HEAP_MB * 1024ULL * 1024;
    const size_t ints = bytes / sizeof(int);

    if (TEST == 0) {
        vector<int> v(ints);
        sink = v.back();
    } // 5c00ed
    else if (TEST == 1) local_array_test();
    else if (TEST == 2) sink = finite_recursion(REC_DEPTH
);
    else if (TEST == 3) {
```

```
vector<int> v;
for (size_t i = 0; i < ints; i++) v.push_back(i);
sink = v.back();
} // f7545b
else if (TEST == 4) {
    queue<int> q;
    for (size_t i = 0; i < ints; i++) q.push(i);
    sink = q.back();
} // bbd9ce
else if (TEST == 5) {
    int a[10] = {};
    volatile size_t index = BAD_INDEX;
    sink = a[index];
} // 5e74db
else if (TEST == 6) infinite_recursion(0);
cout << sink << '\n';
} // baace6
```

```
source.py
Description: Generates a large C++ file to test submission source-size
limits.
Usage: python3 SourceLimitGen.py 100000 > big.cpp
20L, 1m50s
```

```
import sys

N = int(sys.argv[1]) if len(sys.argv) > 1 else 100000

size = 0

def emit(s):
    global size
    sys.stdout.write(s)
    size += len(s.encode())

emit("#include <bits/stdc++.h>\n")
emit("using namespace std;\n")
emit(f"long long a[{N}];\n")
emit("int main() {\n")
for i in range(N): emit(f"a[{i}] = {i};\n")
emit(f"cout << a[{N - 1}] << '\n';\n")
emit("}\n")

print(f"generated {size} bytes ({size / 1024:.1f} KiB)"
, file=sys.stderr)
```